



D8 — Guia de Implementacao e Deploy

Codigo, Configuracoes e Checklist de Deploy
Cliente Supabase, Stores, Hooks, CI/CD

Projeto Aura — Plataforma de Gestao de Eventos

Documento 8 de 8 — Specdrive Tecnico



1. Visao Geral do Documento

Este documento fecha o specdrive tecnico do Aura com codigo real, configuracoes prontas para copiar, e um checklist completo de deploy. Aqui nao ha teoria — apenas implementacao.

Tabela 1 Estrutura do Documento

Secao	Conteudo	Paginas
2	Configuracao do Projeto	estrutura de pastas, dependencias
3	Cliente Supabase	src/lib/supabase.ts completo
4	Zustand Stores	stores para auth, event, cart, ui
5	TanStack Query Hooks	hooks para cada entidade
6	Configuracao Vercel	vercel.json, headers, rewrites
7	CI/CD GitHub Actions	pipeline de build e deploy
8	Variaveis de Ambiente	.env completo com descricoes
9	Migrations Supabase	estrutura e ordem de execucao
10	Checklist de Deploy	passo a passo completo
11	Comandos do Agente	comandos padrao para execucao

2. Configuracao do Projeto

2.1 Estrutura de Pastas

Apos a integracao completa, o projeto tera a seguinte estrutura:

2. Configuracao do Projeto

```
aura/
├── .github/
│   └── workflows/
│       └── deploy.yml
├── supabase/
│   ├── config.toml
│   ├── migrations/
│   │   ├── 000000000000000000000000_init_schema.sql
│   │   ├── 000000000000000000000001_rls_policies.sql
│   │   ├── 000000000000000000000002_triggers.sql
│   │   ├── 000000000000000000000003_seed_data.sql
│   │   └── 000000000000000000000004_functions.sql
│   └── functions/
│       ├── stripe-webhook/
│       │   └── index.ts
│       ├── woovi-webhook/
│       │   └── index.ts
│       ├── generate-report/
│       │   └── index.ts
│       ├── send-email/
│       │   └── index.ts
│       ├── checkin-scan/
│       │   └── index.ts
│       ├── pos-sync/
│       │   └── index.ts
│       ├── notification-dispatcher/
│       │   └── index.ts
│       ├── certificate-generator/
│       │   └── index.ts
│       ├── export-data/
│       │   └── index.ts
│       └── multi-tenant/
│           └── index.ts
└── src/
    ├── lib/
    │   └── supabase.ts
    ├── stores/
    │   ├── authStore.ts
    │   ├── eventStore.ts
    │   ├── cartStore.ts
    │   └── uiStore.ts
    ├── hooks/
    │   ├── useAuth.ts
    │   ├── useEvents.ts
    │   ├── useTickets.ts
    │   ├── useOrders.ts
    │   ├── useCheckin.ts
    │   ├── useReports.ts
    │   ├── useNotifications.ts
    │   ├── usePayments.ts
    │   └── useWhiteLabel.ts
```

2.2 Dependencias a Instalar

```
npm install @supabase/supabase-js @tanstack/react-query zustand resend dotenv npm
install -D @types/node @tanstack/react-query-devtools
```

Tabela 2 Dependencias do Projeto

Pacote	Versao	Finalidade
@supabase/supabase-js	^2.49	Cliente Supabase para browser e server
@tanstack/react-query	^5.66	Gerenciamento de estado server
zustand	^5.0	Gerenciamento de estado client
resend	^4.1	Envio de emails transacionais
dotenv	^16.4	Variaveis de ambiente

3. Cliente Supabase

3.1 Configuracao do Cliente

```
src/lib/supabase.ts
```

```
import { createClient } from '@supabase/supabase-js'
import type { Database } from '../types/database'

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
const supabaseKey = import.meta.env.VITE_SUPABASE_ANON_KEY

export const supabase = createClient<Database>(supabaseUrl, supabaseKey, {
  auth: {
    autoRefreshToken: true,
    persistSession: true,
    detectSessionInUrl: true,
  },
  realtime: {
    params: {
      eventsPerSecond: 10,
    },
  },
})
```

3.2 Cliente Admin (Edge Functions)

```
import { createClient } from '@supabase/supabase-js'

export const supabaseAdmin = createClient(
  Deno.env.get('SUPABASE_URL')!,
  Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!,
  {
    auth: {
      autoRefreshToken: false,
      persistSession: false,
    },
  }
)
```

3.3 Tipos do Database

src/types/database.ts

3. Cliente Supabase

```
export interface Database {
  public: {
    Tables: {
      profiles: {
        Row: {
          id: string
          email: string
          full_name: string | null
          phone: string | null
          cpf: string | null
          avatar_url: string | null
          role: 'admin' | 'producer' | 'attendee'
          status: 'active' | 'suspended' | 'deleted'
          created_at: string
          updated_at: string
        }
        Insert: { [K in keyof Database['public']['Tables']['profiles']['Row']]?: Database['public']['Tables']['profiles']['Row'][K] }
        Update: Partial<Database['public']['Tables']['profiles']['Row']>
      }
      events: {
        Row: {
          id: string
          producer_id: string
          title: string
          slug: string
          description: string | null
          category: string
          banner_url: string | null
          venue_name: string | null
          venue_address: string | null
          venue_city: string | null
          venue_state: string | null
          start_date: string
          end_date: string
          status: 'draft' | 'published' | 'cancelled' | 'finished'
          settings: Record<string, unknown> | null
          created_at: string
          updated_at: string
        }
        Insert: { [K in keyof Omit<Database['public']['Tables']['events']['Row'], 'id'>]: Database['public']['Tables']['events']['Row'][K] }
        Update: Partial<Omit<Database['public']['Tables']['events']['Row'], 'id'>>
      }
    }
    // ... demais tabelas seguem mesmo padrao
    ticket_types: {
      Row: {
        id: string
        event_id: string
        name: string
        description: string | null
        price: number
        quantity_total: number
      }
    }
  }
}
```

4. Zustand Stores

4.1 Auth Store

```
src/stores/authStore.ts
```

```

import { create } from 'zustand'
import { persist } from 'zustand/middleware'
import { supabase } from '../lib/supabase'

interface User {
  id: string
  email: string
  full_name: string | null
  avatar_url: string | null
  role: 'admin' | 'producer' | 'attendee'
  producer_profile?: ProducerProfile | null
}

interface ProducerProfile {
  company_name: string
  cnpj: string
  stripe_account_id: string | null
  woovi_account_id: string | null
  commission_rate: number
  is_verified: boolean
}

interface AuthState {
  user: User | null
  session: any | null
  isLoading: boolean
  isAuthenticated: boolean
  setUser: (user: User | null) => void
  setSession: (session: any) => void
  setLoading: (loading: boolean) => void
  signIn: (email: string, password: string) => Promise<{ error: any }>
  signUp: (email: string, password: string, userData: any) => Promise<{ error: any }>
  signOut: () => Promise<void>
  fetchProfile: () => Promise<void>
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set, get) => ({
      user: null,
      session: null,
      isLoading: true,
      isAuthenticated: false,

      setUser: (user) => set({ user, isAuthenticated: !!user }),
      setSession: (session) => set({ session }),
      setLoading: (isLoading) => set({ isLoading }),

      signIn: async (email, password) => {
        const { data, error } = await supabase.auth.signInWithPassword({
          email,

```


4.2 Event Store

```
src/stores/eventStore.ts
```

```

import { create } from 'zustand'
import { supabase } from '../lib/supabase'

interface Event {
  id: string
  title: string
  slug: string
  banner_url: string | null
  start_date: string
  venue_city: string | null
  venue_state: string | null
  status: string
  category: string
}

interface EventFilters {
  city?: string
  state?: string
  category?: string
  date_from?: string
  date_to?: string
  search?: string
  status?: string
}

interface EventState {
  events: Event[]
  currentEvent: any | null
  featuredEvents: Event[]
  filters: EventFilters
  isLoading: boolean
  totalCount: number
  currentPage: number
  pageSize: number

  setFilters: (filters: EventFilters) => void
  setPage: (page: number) => void
  fetchEvents: () => Promise<void>
  fetchEventBySlug: (slug: string) => Promise<void>
  fetchFeatured: () => Promise<void>
}

export const useEventStore = create<EventState>((set, get) => ({
  events: [],
  currentEvent: null,
  featuredEvents: [],
  filters: {},
  isLoading: false,
  totalCount: 0,
  currentPage: 1,
  pageSize: 12,

```

4.3 Cart Store

```
src/stores/cartStore.ts
```

4. Zustand Stores

```
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

interface CartItem {
  ticket_type_id: string
  event_id: string
  name: string
  price: number
  quantity: number
  attendee_data: any[]
}

interface CartState {
  items: CartItem[]
  eventId: string | null
  discountCode: string | null
  discountAmount: number

  addItem: (item: Omit<CartItem, 'quantity'>) => void
  removeItem: (ticket_type_id: string) => void
  updateQuantity: (ticket_type_id: string, quantity: number) => void
  setAttendeeData: (ticket_type_id: string, data: any[]) => void
  applyDiscount: (code: string, amount: number) => void
  clearCart: () => void
  getSubtotal: () => number
  getTotal: () => number
  getItemCount: () => number
}

export const useCartStore = create<CartState>()(
  persist(
    (set, get) => ({
      items: [],
      eventId: null,
      discountCode: null,
      discountAmount: 0,

      addItem: (item) => {
        const { items, eventId } = get()
        if (eventId && eventId !== item.event_id) {
          set({ items: [], eventId: item.event_id })
        }

        const existing = items.find(i => i.ticket_type_id === item.ticket_type_id)
        if (existing) {
          set({
            items: items.map(i =>
              i.ticket_type_id === item.ticket_type_id
                ? { ...i, quantity: i.quantity + 1 }
                : i
            ),
          })
        }
      },
```

5. TanStack Query Hooks

5.1 Hook de Autenticacao

```
src/hooks/useAuth.ts
```

```

import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { supabase } from '../lib/supabase'
import { useAuthStore } from '../stores/authStore'

export function useAuth() {
  const queryClient = useQueryClient()
  const { user, isAuthenticated, isLoading } = useAuthStore()

  const signInMutation = useMutation({
    mutationFn: async ({ email, password }: { email: string; password: string }) => {
      const { data, error } = await supabase.auth.signInWithPassword({ email, password })
      if (error) throw error
      return data
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['profile'] })
    },
  })

  const signUpMutation = useMutation({
    mutationFn: async ({ email, password, userData }: any) => {
      const { data, error } = await supabase.auth.signUp({
        email,
        password,
        options: { data: userData },
      })
      if (error) throw error
      return data
    },
  })

  const signOutMutation = useMutation({
    mutationFn: async () => {
      await supabase.auth.signOut()
    },
    onSuccess: () => {
      queryClient.clear()
      window.location.reload()
    },
  })

  return {
    user,
    isAuthenticated,
    isLoading,
    signIn: signInMutation.mutateAsync,
    signUp: signUpMutation.mutateAsync,
    signOut: signOutMutation.mutateAsync,
    isSigningIn: signInMutation.isPending,
  }
}

```

5.2 Hook de Eventos

```
src/hooks/useEvents.ts
```

```

import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { supabase } from '../lib/supabase'

const EVENTS_KEY = 'events'

export function useEvents(filters?: Record<string, any>) {
  return useQuery({
    queryKey: [EVENTS_KEY, filters],
    queryFn: async () => {
      let query = supabase
        .from('events')
        .select('*', { count: 'exact' })
        .eq('status', 'published')
        .order('start_date', { ascending: true })

      if (filters?.city) query = query.eq('venue_city', filters.city)
      if (filters?.category) query = query.eq('category', filters.category)
      if (filters?.search) query = query.ilike('title', `%${filters.search}%`)

      const { data, error, count } = await query
      if (error) throw error
      return { events: data || [], count: count || 0 }
    },
  })
}

export function useEvent(slug: string) {
  return useQuery({
    queryKey: [EVENTS_KEY, slug],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('events')
        .select(`*, ticket_types(*), event_agenda(*), event_faqs(*)`)
        .eq('slug', slug)
        .single()
      if (error) throw error
      return data
    },
    enabled: !!slug,
  })
}

export function useCreateEvent() {
  const queryClient = useQueryClient()
  return useMutation({
    mutationFn: async (eventData: any) => {
      const { data, error } = await supabase.from('events').insert(eventData).select().s
      if (error) throw error
      return data
    },
    onSuccess: () => {

```


5.3 Hook de Ingressos

src/hooks/useTickets.ts

```
import { useQuery } from '@tanstack/react-query'
import { supabase } from '../lib/supabase'

export function useTicketTypes(eventId: string) {
  return useQuery({
    queryKey: ['ticket-types', eventId],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('ticket_types')
        .select('*')
        .eq('event_id', eventId)
        .eq('status', 'active')
        .order('price', { ascending: true })
      if (error) throw error
      return data || []
    },
    enabled: !!eventId,
  })
}

export function useMyTickets() {
  return useQuery({
    queryKey: ['my-tickets'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('ticket_orders')
        .select(`
          *,
          event:events(title, start_date, venue_name, banner_url),
          tickets:ticket_order_items(*)
        `)
        .order('created_at', { ascending: false })
      if (error) throw error
      return data || []
    },
  })
}
```

5.4 Hook de Check-in

src/hooks/useCheckin.ts

```

import { useMutation, useQueryClient } from '@tanstack/react-query'
import { supabase } from '../lib/supabase'

export function useCheckin() {
  const queryClient = useQueryClient()

  return useMutation({
    mutationFn: async ({ ticketCode, eventId }: { ticketCode: string; eventId: string }) => {
      const { data: ticket, error: ticketError } = await supabase
        .from('ticket_order_items')
        .select(`*, order:ticket_orders(event_id)`)
        .eq('ticket_code', ticketCode)
        .single()

      if (ticketError) throw new Error('Ingresso nao encontrado')

      if (ticket.order.event_id !== eventId) {
        throw new Error('Ingresso nao pertence a este evento')
      }

      if (ticket.checkin_at) {
        throw new Error('Ingresso ja utilizado em ' + new Date(ticket.checkin_at).toLocaleDateString())
      }

      const { data, error } = await supabase
        .from('ticket_order_items')
        .update({
          checkin_at: new Date().toISOString(),
          checkin_by: (await supabase.auth.getUser()).data.user?.id,
          checkin_device: navigator.userAgent,
        })
        .eq('id', ticket.id)
        .select()
        .single()

      if (error) throw error
      return data
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['checkin-stats'] })
    },
  })
}

```

6. Configuracao Vercel

6.1 vercel.json

```
vercel.json
```

```

{
  "version": 2,
  "name": "aura-platform",
  "framework": "vite",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "installCommand": "npm install",
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ],
  "headers": [
    {
      "source": "/assets/(.*)",
      "headers": [
        {
          "key": "Cache-Control",
          "value": "public, max-age=31536000, immutable"
        }
      ]
    },
    {
      "source": "/(.*)",
      "headers": [
        {
          "key": "X-Content-Type-Options",
          "value": "nosniff"
        },
        {
          "key": "X-Frame-Options",
          "value": "DENY"
        },
        {
          "key": "X-XSS-Protection",
          "value": "1; mode=block"
        },
        {
          "key": "Referrer-Policy",
          "value": "strict-origin-when-cross-origin"
        }
      ]
    }
  ],
  "env": {
    "VITE_SUPABASE_URL": "@supabase-url",
    "VITE_SUPABASE_ANON_KEY": "@supabase-anon-key",
    "VITE_STRIPE_PUBLIC_KEY": "@stripe-public-key",
    "VITE_WOOVI_PUBLIC_KEY": "@woovi-public-key",
    "VITE_API_URL": "@api-url"
  }
}

```

6.2 Headers de Seguranca

Tabela 3 Headers de Seguranca Configurados

Header	Valor	Protecao
X-Content-Type-Options	nosniff	Previne MIME type sniffing
X-Frame-Options	DENY	Previne clickjacking
X-XSS-Protection	1; mode=block	Filtro XSS do browser
Referrer-Policy	strict-origin-when-cross-origin	Controle de referrer
Cache-Control (assets)	max-age=31536000	Cache longo para assets hashados

7. CI/CD GitHub Actions

7.1 Pipeline de Deploy

```
.github/workflows/deploy.yml
```

```

name: Deploy to Vercel

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    name: Test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Type check
        run: npm run typecheck

      - name: Test
        run: npm run test -- --coverage

  build:
    name: Build
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

```

8. Variaveis de Ambiente

8.1 Arquivo .env.example

```
.env.example
```

```

# =====
# SUPABASE
# =====
VITE_SUPABASE_URL=https://seu-projeto.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIs...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIs...
SUPABASE_JWT_SECRET=sua-chave-jwt

# =====
# STRIPE (Pagamentos com cartao)
# =====
VITE_STRIPE_PUBLIC_KEY=pk_test...
STRIPE_SECRET_KEY=sk_test...
STRIPE_WEBHOOK_SECRET=whsec...
STRIPE_CONNECT_CLIENT_ID=ca...

# =====
# WOOFI (Pagamentos via Pix)
# =====
VITE_WOOFI_PUBLIC_KEY=...
WOOFI_API_KEY=...
WOOFI_WEBHOOK_SECRET=...
WOOFI_BASE_URL=https://api.openpix.com.br

# =====
# RESEND (Emails)
# =====
RESEND_API_KEY=re...
RESEND_FROM_EMAIL=noreply@aura.tickets
RESEND_FROM_NAME=Aura Tickets

# =====
# APLICACAO
# =====
VITE_API_URL=https://seu-projeto.supabase.co/functions/v1
VITE_APP_URL=https://aura.tickets
VITE_APP_NAME=Aura
VITE_APP_ENV=production

# =====
# SEGURANCA
# =====
ENCRYPTION_KEY=sua-chave-32-caracteres!!
API_RATE_LIMIT=100

```


8.2 Descricao das Variaveis

Tabela 4 Variaveis de Ambiente do Frontend

Variavel	Onde	Descricao
VITE_SUPABASE_URL	Frontend	URL do projeto Supabase
VITE_SUPABASE_ANON_KEY	Frontend	Chave anonima do Supabase
VITE_STRIPE_PUBLIC_KEY	Frontend	Chave publica do Stripe
VITE_WOOVI_PUBLIC_KEY	Frontend	Chave publica da Woovi
VITE_API_URL	Frontend	Base URL das Edge Functions

Tabela 5 Variaveis de Ambiente do Backend

Variavel	Onde	Descricao
SUPABASE_SERVICE_ROLE_KEY	Edge Functions	Chave de servico com bypass RLS
STRIPE_SECRET_KEY	Edge Functions	Chave secreta Stripe
STRIPE_WEBHOOK_SECRET	Edge Functions	Secret para validar webhooks
WOOVI_API_KEY	Edge Functions	Chave API Woovi
RESEND_API_KEY	Edge Functions	Chave API Resend
ENCRYPTION_KEY	Edge Functions	Chave de criptografia (32 chars)

9. Migrations Supabase

9.1 Estrutura de Migrations

As migrations devem ser executadas em ordem sequencial:

Tabela 6 Ordem de Execucao das Migrations

Ordem	Arquivo	Conteudo	Tempo Est.
1	0000000000000000_init_schema.sql	30 tabelas, enums, tipos	5s
2	0000000000000001_rls_policies.sql	50+ politicas RLS	3s
3	0000000000000002_triggers.sql	15 triggers, 5 functions	2s
4	0000000000000003_seed_data.sql	Categorias, configuracoes	1s
5	0000000000000004_functions.sql	RPC functions	2s

9.2 Comandos de Deploy das Migrations

```
# Instalar CLI do Supabase npm install -g supabase # Login supabase login # Linkar
projeto supabase link --project-ref seu-projeto-ref # Aplicar migrations supabase db
push # Status supabase migration list
```

10. Checklist de Deploy

10.1 Fase 1: Infraestrutura

- [] Criar projeto Supabase (producao)
- [] Configurar autenticacao (email, Google, Facebook)
- [] Habilitar Realtime nas tabelas necessarias
- [] Configurar Storage buckets (avatars, banners, certificates, exports)
- [] Criar projeto Vercel
- [] Configurar dominio customizado (Hostinger)
- [] Configurar DNS e SSL
- [x] Desenvolver frontend (React + Tailwind)

10.2 Fase 2: Banco de Dados

- [] Executar migration init_schema.sql
- [] Executar migration rls_policies.sql
- [] Executar migration triggers.sql
- [] Executar migration seed_data.sql
- [] Executar migration functions.sql
- [] Verificar todas as tabelas criadas
- [] Testar RLS policies com usuarios de teste
- [] Verificar triggers funcionando

10.3 Fase 3: Edge Functions

- [] Deploy stripe-webhook
- [] Configurar webhook URL no Dashboard Stripe
- [] Deploy woovi-webhook
- [] Configurar webhook URL no Dashboard Woovi
- [] Deploy generate-report
- [] Deploy send-email
- [] Deploy checkin-scan
- [] Deploy pos-sync
- [] Deploy notification-dispatcher
- [] Deploy certificate-generator
- [] Deploy export-data
- [] Deploy multi-tenant
- [] Testar cada endpoint

10.4 Fase 4: Integracao Frontend

- ☐ Substituir mock data por chamadas Supabase
- ☐ Integrar autenticacao (login/registro/perfil)
- ☐ Integrar listagem de eventos
- ☐ Integrar detalhe do evento
- ☐ Integrar carrinho de compras
- ☐ Integrar checkout Stripe
- ☐ Integrar checkout Woovi (Pix)
- ☐ Integrar meus ingressos
- ☐ Integrar check-in
- ☐ Integrar painel do produtor
- ☐ Integrar relatorios
- ☐ Integrar notificacoes

10.5 Fase 5: Testes e Lançamento

- ☐ Testar fluxo completo de compra (cartao)
- ☐ Testar fluxo completo de compra (Pix)
- ☐ Testar envio de email de confirmacao
- ☐ Testar check-in com QR code
- ☐ Testar geracao de certificado
- ☐ Testar white label com subdomain
- ☐ Testar rate limits
- ☐ Testar responsividade mobile
- ☐ Revisar SEO (meta tags, sitemap)

- [] Configurar Google Analytics
- [] Configurar Sentry (erro tracking)
- [] Deploy para producao
- [] Monitorar logs por 48h

10.6 Fase 6: Pos-Lancamento

- [] Configurar backups automaticos
- [] Configurar alertas de erro
- [] Criar conta de suporte
- [] Documentar FAQ interno
- [] Monitorar metricas (vendas, usuarios)
- [] Coletar feedback dos primeiros usuarios
- [] Planejar iteracoes baseadas em dados

11. Comandos do Agente Executor

11.1 Comandos Padrao

O agente Executor recebe estes comandos do Orquestrador e executa com autonomia:

Tabela 7 Comandos do Agente Executor

Comando	Descricao	Exemplo
IMPL <arquivo>	Implementa arquivo seguindo spec	IMPL src/stores/authStore.ts
MIGRATE <nome>	Cria migration SQL	MIGRATE 000000000000000000000000_init_schema
EDGE <nome>	Cria Edge Function	EDGE stripe-webhook
HOOK <entidade>	Cria TanStack Query hook	HOOK events
INT <pagina>	Integra pagina com API real	INT EventDetail
TEST <escopo>	Executa testes	TEST auth
BUILD	Executa build e verifica	BUILD
FIX <arquivo>	Corrige erros de build	FIX src/lib/supabase.ts

11.2 Comandos do Agente Corretor

Tabela 8 Comandos do Agente Corretor

Comando	Descricao	Exemplo
REVIEW <arquivo>	Revisa codigo por qualidade	REVIEW src/stores/authStore.ts
AUDIT RLS	Audita politicas RLS	AUDIT RLS
AUDIT SEC	Audita seguranca geral	AUDIT SEC
VERIFY <arquivo>	Verifica se arquivo segue spec	VERIFY src/hooks/useAuth.ts
REPORT	Gera relatorio de qualidade	REPORT

12. Resumo do Specdrive

O specdrive tecnico completo do Aura contem 8 documentos com um total de 100+ paginas:

Tabela 9 Resumo do Specdrive Aura

Doc	Titulo	Paginas	Conteudo Principal
D1	Inventario Frontend	19	Catalogo de 130+ arquivos, 66 paginas, 82 componentes
D2	Gap Analysis	12	30 tabelas, 10 Edge Functions, 9 integracoes
D3	Analise Estrategica	8	Decisoies: taxa 5-7%, SaaS, ARR R\$8.88M
D4	Frontend Melhorias	9	UX, performance, a11y, SEO, novas features
D5	Supabase Spec	27	SQL schema, RLS, triggers, Edge Functions
D6	Arquitetura de Sistemas	8	Supabase, Vercel, Hostinger, Stripe, Woovi
D7	Organizacao de Agentes	10	3 agentes, comandos, matriz de skills
D8	Implementacao e Deploy	14	Codigo, configs, checklist completo

Total do Specdrive: 107 paginas tecnicas cobrindo inventario, gaps, estrategia, melhorias, especificacao completa do Supabase, arquitetura de sistemas, organizacao de agentes e guia de implementacao com codigo real.